Task 3 — Product Catalog with Search & Filter

Overview

Build a product browsing experience with combinable keyword search, category and price filters, sorting, and pagination. This task exercises query composition, pagination metadata, and stateful filter UI.

User Story

> ***As a*** *shopper,* ***I want*** *to search and filter products by keyword, category, and price range,* ***so that*** *I can quickly find items I'm interested in.*

Your Story Point Estimation (To Be Completed by Learner)

Estimate the effort for this user story using a Fibonacci scale (1, 2, 3, 5, 8, 13, ...). Consider complexity, effort, risk, and uncertainty, then justify your reasoning.

Factor	Your Assessment
Complexity *(how hard is the logic?)*	*(fill in)*
Effort *(how much work / how many parts?)*	*(fill in)*
Uncertainty / Risk *(unknowns, dependencies?)*	*(fill in)*
Your Estimate (points)	*(fill in)*
Rationale	*(explain how you arrived at your estimate)*

Acceptance Criteria

- Users can filter by keyword (matches name/description), one or more categories, and a min/max price.
- Filters are **combinable** — applying several narrows results by their intersection.
- Results are paginated (e.g., 10 per page) and return total count + page metadata.
- Results can be sorted (e.g., price ascending/descending, newest).
- An empty result set returns `200` with an empty list and a "no products found" state.
- Invalid parameters (e.g., `minPrice > maxPrice`, negative price) are handled gracefully.

Data Model

Field	Type	Constraints
`id`	UUID / int	Primary key
`name`	string	Required, searchable
`description`	string	Searchable
`category`	string / enum	Indexed
`price`	decimal	≥ 0
`createdAt`	timestamp	Used for "newest" sort

API Contract (Framework Agnostic)

`GET /products?search=&category=&minPrice=&maxPrice=&sort=&page=&pageSize=`

- Success `200`:

```
{
  "items": [ { "id": "...", "name": "...", "price": 19.99, "category": "..." } ],
  "page": 1,
  "pageSize": 10,
  "totalItems": 42,
  "totalPages": 5
}
```

- `category` may repeat for multi-select (e.g., `category=books&category=toys`).
- Errors: `400` for malformed parameters.

Back-End Requirements

- Build the query dynamically from whichever filters are present (all optional).
- Support multi-category selection and combined keyword + price filtering.
- Apply a sensible default sort and allow overriding via `sort`.
- Return pagination metadata (`page`, `pageSize`, `totalItems`, `totalPages`).
- Clamp/validate paging (e.g., `page ≥ 1`, bounded `pageSize`).

Front-End Requirements

- Search box (debounced), category multi-select, and min/max price inputs that update results.
- Sort selector and paginated grid/list with page navigation (prev/next or numbered).
- Loading skeleton/spinner during fetch; clear empty state when no matches.
- Reflect active filters visibly (chips/badges) and allow clearing them.
- Preserve filter state on pagination (changing page keeps filters).

Non-Functional

- Debounce keyword input to avoid excessive requests.

- Response should remain fast with a large catalog (indexing/pagination assumed).

Edge Cases to Handle

- `minPrice > maxPrice` → return empty set (or `400`) without crashing.
- Negative or non-numeric price parameters.
- Page number beyond `totalPages` → return empty items with correct metadata.
- Keyword with special characters (no injection; safe parameterization).

Unit Test Scenarios (Given / When / Then)

#	Given	When	Then
1	Products across categories	Filter by one category	Only matching-category items return
2	Products at various prices	Keyword + price-range filter	Correct intersection is returned
3	42 products, pageSize 10	Request page 2	Items 11–20 and total metadata return
4	`minPrice > maxPrice`	Filter is applied	Empty set returned without error
5	A keyword matching nothing	Search is applied	Empty list + `totalItems` = 0

Deliverables

- Back-end filtered/paginated/sorted product endpoint.
- Front-end catalog UI with search, filters, sort, and pagination.
- Unit tests covering all scenarios above.
- Short README: how to run the app and tests, plus your completed estimation table.